Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

RESQ – Intelligent Disaster Rescue & Emergency Response Simulator
Tagline: "Algorithms That Save Lives."

Student / Team Information

Team Name: *Team ID:*	**RESQ Force** DSCPP-III-2026-T117	
Team member 1 (Team Lead)	*Vanya Rathi – 2510383410* *2510383410@geu.ac.in* *University Roll No.: 2028514* *Phone No.: 9259011570* *Section: DS-2*	
Team member 2	*Akshat Dobhal – 2510370353* *2510370353@geu.ac.in* *University Roll No.: 2028631* *Phone No.: 9389135760* *Section: ML-2*	
Team member 3	*Ayush Kandwal – 2510380235* *2510380235@geu.ac.in* *University Roll No.: 2028405* *Phone No.: 9068120181* *Section: ML-4*	
Team member 4	*Nitin Singh – 2510370229* *2510370229@geu.ac.in* *University Roll No.: 2028786* *Phone No.: 6395245688* *Section: ML-1*	

Project Mentor: Dr. Prabhdeep Singh **Interactions completed:** 02

PHASE-III PROJECT PROGRESS DESCRIPTION

Project Abstract

RESQ is our classroom simulator for organising rescue work after a disaster. The main goal was to decide who should be helped first and how a team should travel when some roads are slow or blocked. Our final version takes victim details and a disaster scenario, places requests into critical or normal queues, checks reachability with BFS and calculates routes with Dijkstra. It then assigns teams or ambulances, reserves hospital beds and records each operation. In the fixed earthquake test, seven calls were logged, six victims were rescued and one unreachable case was left for air support. We also built a separate browser demo with a map, priority filters and an operations dashboard. The two versions share the same logic but do not call each other. RESQ demonstrates algorithms; it is not a live emergency service.

Updated Project Approach and Architecture

Our native solution is split into C data-structure modules and C++ operation classes. The C layer stores victim records in a linked list, uses a circular FIFO queue for routine calls and a binary max-heap for critical calls. A stack keeps operation history. The map is a weighted adjacency matrix. BFS checks which locations can still be reached, and Dijkstra compares travel costs before a unit is dispatched. Insertion sort, quick sort, binary search and linear search help us inspect records.

The C++ layer models Location, Victim, RescueUnit, RescueTeam, Ambulance, Hospital, Disaster and RescueOperation. RescueTeam and Ambulance inherit from RescueUnit and override `describe()`, so the same interface prints the correct resource details. The browser version uses TypeScript and React for the same basic triage and routing ideas, with an SVG map and session-only state. There is no external database, GPS feed or emergency API. We used GCC/G++ with warning flags, Make, GitHub, and comparison against saved output for checking.

Tasks Completed

Task Completed	Team Member
Architecture, triage logic, integration and final documentation	Vanya Rathi (DS-2)
Linked list, FIFO queue, critical max-heap and operation-history stack	Akshat Dobhal (ML-2)
Weighted graph, BFS reachability and Dijkstra route planning	Ayush Kandwal (ML-4)
C++ resource classes, scenario testing, dashboard and reports	Nitin Singh (ML-1)

Challenges/Roadblocks

One early problem was making priority fair without losing the arrival order. We used severity first and kept the earlier call ahead when scores matched. Road blocks caused another issue: a route could become longer or disappear completely. BFS now checks access before dispatch and unreachable requests stay unresolved instead of being marked rescued. Hospital capacity also mattered, so critical cases are sent to trauma care and beds are reduced only after a transfer. Joining the C and C++ parts took care because both sides had to agree on record layout and ownership. We cleaned up the interfaces and reduced the source to five main code files without changing output. The browser demo needed small-screen adjustments so labels and road times could be read on a phone. We still want more varied test scenarios and one shared core for the native and web versions.

Project Outcome/Deliverables

The final outcome is a working classroom prototype in two forms. The native C/C++ program runs a complete earthquake scenario and prints calls, queues, routes, hospital transfers, sorted records, operation history and a mission summary. The separate web demo lets us enter victims, select disaster settings, inspect the priority order, draw Dijkstra routes, dispatch operations and filter the dashboard. The submission also contains simplified source code, a Makefile, README, saved sample output, reports, presentations and a GitHub repository. These deliverables show how the algorithms work; they are not certified for real rescue work.

Progress Overview

The planned classroom prototype is complete. Phase I gave us the design, Phase II produced the main working modules, and Phase III brought the rescue flow, documentation and final checks together. We also simplified the native source and improved the phone layout of the web demo. Nothing from the planned demonstration scope is pending. Larger ideas such as persistent storage, live GPS, more disaster scenarios and a shared native/web core remain future work rather than unfinished claims. We tested one fixed native scenario thoroughly, but we have not run a large performance benchmark or connected the project to any emergency service.

Codebase Information

Repository: `https://github.com/akshat017-beep/RESQ-disaster-response-simulator`

Branch: `main`. The `Code/` folder contains the C/C++ program, Makefile, README and sample output. The `Phase 1/` and `Phase 2/` folders contain earlier submission documents. The source was also simplified into five main code files: `resq.h`, `dsa.c`, `resq.hpp`, `resq.cpp` and `main.cpp`. Its output was checked against the original version and remained unchanged. Code and documents were uploaded through shared commits, so we are not claiming separate commit counts for each member.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Build with warning flags	Pass	GCC C11 and G++ C++17 build completed without warnings.
Critical-first dispatch	Pass	Severity 9, 8 and 7 cases were handled before routine FIFO calls.
BFS and blocked road	Pass	9 of 10 locations were reachable; Hill Colony stayed unresolved.
Dijkstra route table	Pass	Command Centre to Paltan Bazaar cost 7 and Doon Trauma Centre cost 13 simulated minutes.
Complete rescue scenario	Pass	7 calls logged, 6 rescued, 1 unreachable and 4 resources deployed.
Output regression check	Pass	Fresh output matched `sample-output.txt` byte for byte.

Deliverables Progress

Completed: native C/C++ simulator; linked list, queues, heap, stack, sorting and searching; BFS and Dijkstra routing; C++ class model; earthquake demonstration; saved sample output; browser map and operations dashboard; project reports and presentations; GitHub source upload.

Verified: clean native build, repeatable full-scenario output and the results listed above. **Future scope only:** persistent storage, user login, live GPS or emergency-service integration, more test scenarios and one shared core for the native and browser versions. No planned classroom demonstration module is marked pending.